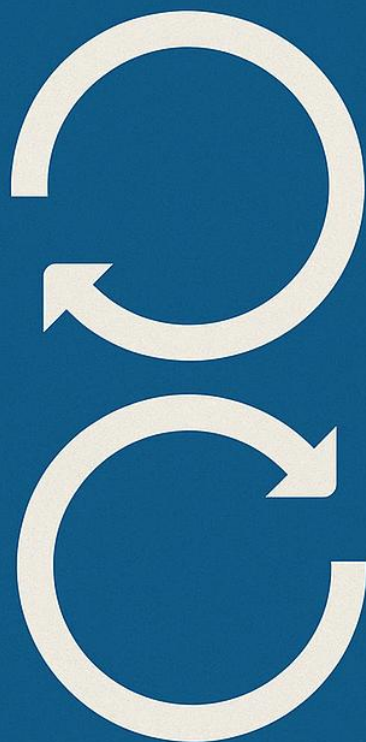


DOMINANDO AS REPETIÇÕES

**UM GUIA COMPLETO SOBRE
ENQUANTO E FAÇA-ENQUANTO**



**INSTRUTOR:
DJALMA BATISTA BARBOSA JUNIOR**

Capítulo 1:

A Necessidade de Repetir: Introdução aos Laços de Repetição

Por que repetir? A automação de tarefas

No coração da computação e da programação reside uma ideia simples, mas poderosa: a capacidade de executar tarefas repetitivas de forma rápida, precisa e incansável. Imagine que precisa de enviar uma mensagem personalizada para cem pessoas. Fazer isso manualmente seria tedioso e propenso a erros.

Um computador, no entanto, pode ser instruído a repetir o processo de "pegar o próximo nome da lista, inserir no modelo da mensagem e enviar" cem vezes, completando a tarefa em segundos. Esta capacidade de automatizar a repetição é o que torna os computadores ferramentas tão transformadoras.

No nosso dia a dia, realizamos inúmeras ações repetitivas sem sequer pensar nelas como um "processo". Lavar uma pilha de pratos, por exemplo, é um algoritmo de repetição: enquanto houver pratos sujos na pia, repete-se a sequência de pegar um prato, ensaboar, enxaguar e colocar para secar.

A tarefa só termina quando a condição "a pia está vazia" é satisfeita. Em programação, formalizamos esta ideia através de estruturas de controle conhecidas como

laços de repetição ou, simplesmente, **loops**.

Apresentando o "Laço" (Loop)

Um laço de repetição é um comando ou um conjunto de comandos que permite que um bloco de código seja executado múltiplas vezes.

Em vez de escrever a mesma instrução dez, cem ou mil vezes, um programador pode usar um laço para instruir o computador a executar essa instrução pelo número de vezes desejado ou até que uma determinada condição seja alcançada.

Esta abordagem não só economiza tempo e esforço, mas também torna o código mais limpo, mais legível e mais fácil de manter.

Vocabulário Essencial

Para navegar no mundo dos laços de repetição, é fundamental dominar alguns termos essenciais que formam a base da sua lógica:

- **Iteração:** Este termo refere-se a uma única execução do bloco de código contido dentro de um laço. Se um laço executa o seu bloco de código cinco vezes, dizemos que ocorreram cinco iterações. Na nossa analogia, cada prato que é completamente lavado corresponde a uma iteração do processo.
- **Condição de Parada:** Esta é a regra lógica que o computador verifica para decidir se deve continuar a executar o laço ou se deve parar. A cada iteração, a condição é testada. Enquanto a condição for satisfeita (geralmente resultando em um valor lógico "verdadeiro"), o laço continua. Quando a condição deixa de ser satisfeita (resultando em "falso"), o laço termina, e o programa continua a executar as instruções seguintes. Para a tarefa de lavar os pratos, a condição de parada é "não há mais pratos sujos na pia".

A Ponte com o Conhecimento Prévio

Os alunos que já estudaram a estrutura condicional SE deram o primeiro passo para entender a lógica de controle de fluxo. A estrutura SE permite que um programa tome uma decisão e execute um bloco de código *uma única vez* se uma condição for verdadeira.

SE (condição é verdadeira) ENTÃO
// Executa este bloco de código uma vez
FIM_SE

Os laços de repetição levam este conceito um passo adiante. Em vez de executar uma vez, eles reavaliam a condição após cada execução do bloco. Pode-se pensar num laço ENQUANTO como uma estrutura SE que, ao final da sua execução, volta ao início para testar a condição novamente.

ENQUANTO (condição é verdadeira) FAÇA
// Executa este bloco de código...
//...e depois volta para testar a condição de novo
FIM_ENQUANTO

A principal diferença conceptual é a introdução de um *estado que muda ao longo do tempo*. Numa estrutura SE, a condição é avaliada num único momento. Num laço, para que ele eventualmente termine, algo dentro do bloco de código deve alterar o estado do programa — ou seja, o valor de uma ou mais variáveis — de forma que a condição de parada seja, em algum momento, alcançada.

Sem esta mudança progressiva, o laço nunca terminaria, criando um problema conhecido como "loop infinito". Esta necessidade de gerir a mudança de estado é o que torna os laços uma ferramenta tão dinâmica e fundamental na programação.

Capítulo 2:

A Estrutura ENQUANTO: O Laço com Teste no Início (Pré-teste)

A Lógica do "Verificar Antes de Agir"

A estrutura ENQUANTO (conhecida como while em muitas linguagens de programação) é um dos tipos mais fundamentais de laços de repetição. A sua característica definidora é que ela opera com uma lógica de **pré-teste**. Isto significa que a condição de parada é verificada

antes de cada iteração, incluindo a primeira.

O fluxo de execução é direto:

1. O programa chega ao início do laço ENQUANTO.
2. A condição lógica é avaliada.
3. Se a condição for **falsa**, o bloco de código dentro do laço é completamente ignorado, e o programa salta para a primeira instrução após o FIM_ENQUANTO.
4. Se a condição for **verdadeira**, o bloco de código é executado na sua totalidade.
5. Após a execução do bloco, o fluxo retorna ao passo 2 para uma nova verificação da condição.

Esta natureza de pré-teste tem uma implicação crucial: se a condição for falsa desde a primeira vez que é avaliada, o bloco de comandos do laço ENQUANTO **nunca será executado**, nem mesmo uma vez.

Sintaxe em Pseudocódigo (Portugol)

A representação da estrutura ENQUANTO em pseudocódigo, como o Portugol, segue um formato padrão e intuitivo que reflete a sua lógica.

```
algoritmo "ExemploEnquanto"  
var  
    // Declaração de variáveis  
inicio  
    // Comandos antes do laço  
  
    ENQUANTO <condição lógica> FAÇA  
        // Bloco de comandos a ser repetido  
        // enquanto a condição for verdadeira.  
        // É essencial que algo aqui dentro  
        // modifique as variáveis da condição.  
    FIM_ENQUANTO  
  
    // Comandos após o laço  
fimalgoritmo
```

A <condição lógica> é uma expressão que resulta num valor booleano (verdadeiro ou falso), como $x < 10$, $\text{resposta} = \text{"S"}$ ou $\text{valorDigitado} \neq 0$.

Representação Visual com Fluxograma

Um fluxograma é uma excelente ferramenta para visualizar o fluxo de controle de um algoritmo. Para a estrutura ENQUANTO, o fluxograma destaca claramente o ciclo de verificação e execução.

O fluxo começa e encontra um losango de decisão, que representa o teste da condição.

- Se a condição avaliada no losango for **falsa** (o caminho "Não"), o fluxo desvia-se do bloco de repetição e continua para o resto do programa.
- Se a condição for **Verdadeira** (o caminho "Sim"), o fluxo entra num retângulo, que representa o bloco de comandos a ser executado. Após a execução completa do bloco, a seta de fluxo retorna para um ponto *antes* do losango de decisão, forçando uma nova avaliação da condição e fechando o ciclo (o "loop").

O Ciclo de Vida de um Laço:

Inicialização, Verificação e Atualização

Para que um laço ENQUANTO funcione corretamente e, crucialmente, termine, ele depende de um padrão de três etapas que pode ser chamado de "Ciclo de Vida do Laço". Compreender este ciclo é a chave para construir laços eficazes e evitar erros.

1. **Inicialização:** Antes de o laço começar, a(s) variável(is) que faz(em) parte da condição de parada deve(m) receber um valor inicial. Sem um ponto de partida conhecido, o primeiro teste da condição seria imprevisível.
2. **Verificação:** Esta é a condição lógica no cabeçalho do ENQUANTO. Ela é o portão de entrada para cada iteração.
3. **Atualização:** Dentro do bloco de comandos do laço, deve haver pelo menos uma instrução que modifique a(s) variável(is) da condição, aproximando o estado do programa de um ponto em que a condição se tornará falsa.

A falha em qualquer uma destas etapas levará a um comportamento incorreto do laço. A ausência da atualização, por exemplo, é a causa mais comum de loops infinitos.

Exemplo Prático 1: O Padrão Contador

Um **contador** é uma variável, geralmente numérica, usada para controlar o número de vezes que um laço é executado. O exemplo mais simples é um algoritmo que escreve os números de 1 a 5 na tela.

algoritmo "ContadorSimples"

var

contador: inteiro

inicio

// 1. INICIALIZAÇÃO

contador <- 1

// 2. VERIFICAÇÃO

ENQUANTO contador <= 5 FACA

```
escreval(contador)
```

```
// 3. ATUALIZAÇÃO
```

```
contador <- contador + 1
```

```
FIM_ENQUANTO
```

```
escreval("Fim da contagem!")
```

```
fimalgoritmo
```

Neste exemplo:

- `contador <- 1` é a **inicialização**. O nosso ponto de partida é o número 1.
- `ENQUANTO contador <= 5 FACA` é a **verificação**. O laço continuará enquanto o valor do contador for menor ou igual a 5.
- `contador <- contador + 1` é a **atualização**. A cada iteração, o valor do contador é incrementado em 1. Esta linha é vital; sem ela, contador seria sempre 1, a condição `1 <= 5` seria sempre verdadeira, e o programa imprimiria "1" infinitamente.

Exemplo Prático 2: O Padrão Acumulador

Um **acumulador** é uma variável usada para "coleccionar" ou "acumular" valores ao longo de múltiplas iterações. É frequentemente usado para calcular somas, produtos ou para concatenar textos.

Vamos construir um algoritmo que soma vários números fornecidos pelo utilizador. O programa deve parar de pedir números quando o utilizador digitar o valor 0, que funcionará como um "sentinela" (um sinal para parar).

```
algoritmo "SomaComSentinela"
```

```
var
```

```
    soma, valorDigitado: real
```

```
inicio
```

```
    // 1. INICIALIZAÇÃO do acumulador e da variável de controle
```

```
    soma <- 0
```

```
    escreva("Digite um valor para somar (ou 0 para sair): ")
```

```
    leia(valorDigitado)
```



```

// 2. VERIFICAÇÃO
ENQUANTO valorDigitado <> 0 FACA
  // ATUALIZAÇÃO do acumulador
  soma <- soma + valorDigitado

  escreval("Soma parcial: ", soma)

  // ATUALIZAÇÃO da variável de controle
  escreva("Digite o próximo valor (ou 0 para sair): ")
  leia(valorDigitado)
FIM_ENQUANTO

  escreval("A soma total dos valores é: ", soma)
finalgoritmo

```

Neste caso:

- soma <- 0 inicializa o nosso acumulador. É fundamental começar com zero para uma soma.
- A primeira instrução leia(valorDigitado) é parte da inicialização da variável de controle. Precisamos de um valor inicial para que a primeira verificação valorDigitado <> 0 possa ser feita.
- ENQUANTO valorDigitado <> 0 FACA é a verificação.
- soma <- soma + valorDigitado atualiza o acumulador a cada iteração.
- A segunda instrução leia(valorDigitado), dentro do laço, é a atualização da variável de controle, preparando o laço para a próxima verificação.

Capítulo 3:

A Estrutura FAÇA-ENQUANTO: O Laço com Teste no Final (Pós-teste)

A Lógica do "Agir Primeiro, Perguntar Depois"

Em contraste com a estrutura ENQUANTO, o laço FAÇA-ENQUANTO (ou REPITA-ATÉ) opera com uma lógica de **pós-teste**. A sua característica fundamental é que o bloco de comandos é executado primeiro, e só depois a condição de parada é verificada.

O fluxo de execução é o seguinte:

1. O programa chega ao início do laço FAÇA.
2. O bloco de comandos dentro do laço é executado na sua totalidade.
3. Ao final do bloco, a condição lógica associada ao ENQUANTO é avaliada.
4. Se a condição for **verdadeira**, o fluxo retorna ao passo 2, e o bloco é executado novamente.
5. Se a condição for **falsa**, o laço termina, e o programa continua a sua execução normal.

A consequência mais importante desta abordagem é que o bloco de comandos de um laço FAÇA-ENQUANTO tem a **garantia de ser executado pelo menos uma vez**, independentemente do resultado inicial da condição.⁵

Sintaxe em Pseudocódigo (Portugol)

Existem duas variações comuns de sintaxe para laços pós-testados em pseudocódigo.

A primeira é a REPITA-ATÉ. Nesta estrutura, o laço continua a repetir o bloco de comandos *até que* a condição se torne verdadeira. Ou seja, ele repete enquanto a condição é **falsa**.

REPITA

// Bloco de comandos

ATE <condição lógica se tornar verdadeira>

A segunda, e mais alinhada com a sintaxe de muitas linguagens de programação modernas (como C++, Java, C#), é a FAÇA-ENQUANTO. Nesta estrutura, o laço continua a repetir o bloco de comandos *enquanto* a condição for verdadeira, de forma análoga ao laço ENQUANTO. A única diferença é o momento do teste. Para manter a coerência e facilitar a transição para linguagens de programação reais, focaremos nesta segunda variante.

algoritmo "ExemploFacaEnquanto"

var

// Declaração de variáveis

inicio

// Comandos antes do laço

FAÇA

// Bloco de comandos que será

// executado pelo menos uma vez.

ENQUANTO <condição lógica for verdadeira>

// Comandos após o laço

fimalgoritmo

Representação Visual com Fluxograma

O fluxograma da estrutura FAÇA-ENQUANTO ilustra visualmente a sua lógica "agir primeiro, perguntar depois". O fluxo entra diretamente no retângulo que representa o bloco de comandos.

Somente após sair do retângulo é que o fluxo encontra o losango de decisão onde a condição é testada. Se a condição for verdadeira, uma seta retorna o fluxo para o início do bloco de comandos; se for falsa, o fluxo prossegue para fora do laço.

Exemplo Prático 1: Menu de Opções

Um dos casos de uso mais clássicos para a estrutura FAÇA-ENQUANTO é a implementação de menus interativos. O programa precisa de mostrar as opções ao utilizador *pelo menos uma vez* para que ele saiba o que pode escolher. O laço continua a exibir o menu e a processar a escolha até que o utilizador selecione a opção de sair.

algoritmo "MenuInterativo"

var

opcao: inteiro

inicio

FAÇA

escreval("=====")

escreval(" M E N U ")

escreval("=====")

escreval("1 - Cadastrar Produto")

escreval("2 - Listar Produtos ")

escreval("0 - Sair ")

escreval("=====")

escreva("Escolha uma opção: ")

leia(opcao)

// Aqui viria a lógica para tratar cada caso (usando SE ou ESCOLHA-CASO)

ENQUANTO opcao <> 0

escreval("Programa finalizado.")

fimalgoritmo

Neste exemplo, o bloco que exibe o menu e lê a opção do utilizador é executado garantidamente na primeira vez. Só depois é que a condição `opcao <> 0` é verificada para decidir se o menu deve ser exibido novamente.

Exemplo Prático 2: Validação de Entrada de Dados

Talvez o cenário mais importante e elegante para o uso do FAÇA-ENQUANTO seja a validação de dados inseridos pelo utilizador. Imagine que precisa de garantir que o utilizador digite uma nota que esteja no intervalo de 0 a 10. O processo lógico é:

1. Pedir e ler a nota.
2. Verificar se a nota é válida.
3. Se for inválida, voltar ao passo 1.

Este fluxo encaixa-se perfeitamente na estrutura FAÇA-ENQUANTO, pois é necessário primeiro obter o dado (FAÇA) para depois poder validá-lo (ENQUANTO).

```
algoritmo "ValidarNota"
var
  nota: real
inicio
  FAÇA
    escreva("Digite uma nota entre 0 e 10: ")
    leia(nota)

    se (nota < 0) ou (nota > 10) entao
      escreval("Valor inválido! Tente novamente.")
    fim_se

  ENQUANTO (nota < 0) ou (nota > 10)

  escreval("Nota válida registrada: ", nota)
finalgoritmo
```

Este código é notavelmente limpo e direto. A instrução `leia(nota)` aparece apenas uma vez. Se tentássemos resolver este mesmo problema com um laço ENQUANTO, encontraríamos uma pequena "inelegância" de código.

Seria necessário ler a nota uma vez *antes* do laço para a primeira verificação, e depois novamente *dentro* do laço para as tentativas subsequentes, resultando em código duplicado. A estrutura

FAÇA-ENQUANTO elimina esta redundância, tornando o código mais conciso e menos propenso a erros, pois qualquer alteração na lógica de leitura só precisa de ser feita num único lugar.

Capítulo 4:

ENQUANTO vs. FAÇA-ENQUANTO: O Duelo dos Laços

A escolha entre ENQUANTO e FAÇA-ENQUANTO não é uma questão de preferência pessoal, mas sim uma decisão de design baseada na lógica do problema a ser resolvido. Ambas as estruturas são capazes de resolver os mesmos problemas, mas uma delas geralmente oferece uma solução mais natural, legível e eficiente para um determinado cenário.

A Pergunta Crucial: "O bloco DEVE executar pelo menos uma vez?"

Este é o principal critério para decidir qual laço utilizar. Ao analisar um problema que requer repetição, o programador deve fazer a si mesmo esta pergunta fundamental:

- **Se a resposta for "Sim, o bloco de comandos precisa de ser executado obrigatoriamente pelo menos uma vez", então a estrutura FAÇA-ENQUANTO é quase sempre a escolha mais adequada.** Exemplos clássicos incluem a exibição de um menu de opções (o utilizador precisa de ver o menu para poder escolher) e a validação de entrada de dados (é preciso receber o dado para depois validá-lo).
- **Se a resposta for "Não, é possível que o bloco de comandos nunca seja executado" ou "Talvez, depende de uma condição inicial", então a estrutura ENQUANTO é a escolha mais segura e apropriada.** Exemplos incluem processar os itens de uma lista (a lista pode estar vazia), realizar uma ação enquanto uma condição externa persistir (ex: enquanto houver saldo na conta), ou simplesmente contar até um certo número (se o limite inicial for menor que o ponto de partida, o laço não executa).

Análise Lado a Lado

Vamos revisitar dois dos nossos exemplos para solidificar esta distinção.

1. Contar de 1 a 5:

- Com ENQUANTO: A lógica é natural. Inicializamos contador $\leftarrow 1$. A condição contador ≤ 5 é verificada. Como é verdadeira, o laço começa. Se, por algum motivo, inicializássemos contador $\leftarrow 6$, a condição $6 \leq 5$ seria falsa desde o início, e o laço corretamente não executaria nenhuma vez.
- Com FAÇA-ENQUANTO: Seria possível, mas a lógica ficaria estranha. Se inicializássemos contador $\leftarrow 6$, o laço executaria uma vez, imprimiria "6", incrementaria para 7, e só então a condição $7 \leq 5$ seria verificada como falsa. O resultado estaria incorreto.

2. Validar uma nota (0-10):

- Com ENQUANTO: Como discutido, isto requer uma leitura *antes* do laço e outra *dentro* do laço, duplicando código.
- Com FAÇA-ENQUANTO: A solução é elegante e sem redundância. Uma única leitura dentro do laço é suficiente, pois a execução garantida na primeira vez permite obter o valor que será validado na condição ao final da iteração.

Tabela Comparativa

A tabela seguinte resume as diferenças fundamentais entre as duas estruturas, servindo como um guia de referência rápida para a tomada de decisão algorítmica.

Característica	Estrutura ENQUANTO	Estrutura FAÇA-ENQUANTO
Momento do Teste	No início da iteração (Pré-teste)	No final da iteração (Pós-teste)
Execução Mínima	0 (zero) vezes. Pode nunca executar se a condição inicial for falsa.	1 (uma) vez. O bloco de comandos é sempre executado no mínimo uma vez.
Fluxo Lógico	1. Testa a condição. 2. Se for Verdadeira, executa o bloco.	1. Executa o bloco. 2. Testa a condição.
Cenário Ideal	Processar dados que podem não existir (ex: ficheiros, listas). Repetir ações baseadas num estado que pode já ser final. Contagens e acumulações com condições de entrada estritas.	Menus interativos. Validação de entrada de dados. Situações onde a primeira ação é necessária para avaliar a condição de continuação.

Em suma, a escolha correta não só torna o código funcional, mas também o torna mais expressivo, comunicando claramente a intenção do programador e a lógica inerente ao problema.

Capítulo 5:

Armadilhas Comuns e Boas Práticas

Escrever laços de repetição é uma habilidade essencial, mas também é uma área onde os programadores iniciantes frequentemente encontram dificuldades. A armadilha mais comum e frustrante é o "loop infinito". Compreender as suas causas e como evitá-lo é um passo crucial para se tornar um programador proficiente.

O Temido Loop Infinito

Um loop infinito é um laço cuja condição de parada nunca é alcançada. Como resultado, o programa fica preso, executando o mesmo bloco de código indefinidamente, consumindo recursos do sistema (como processamento e memória) até que seja forçado a parar externamente (por exemplo, fechando o programa ou, em ambientes de desenvolvimento, usando um atalho como CTRL + C). Na grande maioria dos casos, um loop infinito é o resultado de um erro de lógica na programação.

Causa Raiz: A Falha na Atualização

A causa mais comum de um loop infinito é o esquecimento ou a implementação incorreta da **etapa de atualização** do "Ciclo de Vida do Laço". Se a variável que controla a condição nunca é alterada dentro do laço, ou é alterada de uma forma que nunca a levará a tornar a condição falsa, o laço nunca terminará.

Considere esta versão defeituosa do nosso algoritmo contador:

```
// ATENÇÃO: CÓDIGO COM ERRO (LOOP INFINITO)
algoritmo "ContadorInfinito"
var
    contador: inteiro
inicio
    contador <- 1

    ENQUANTO contador <= 5 FAÇA
        escreval("O valor do contador é: ", contador)
        // A linha de atualização foi esquecida!
        // contador <- contador + 1 <-- ESTA LINHA FALTA
    FIM_ENQUANTO

    escreval("Esta mensagem nunca será exibida.")
fimalgoritmo
```

Neste código, a variável contador é inicializada com 1. A condição $1 \leq 5$ é verdadeira, então o laço começa. O programa imprime "O valor do contador é: 1". Como não há nenhuma linha para incrementar o contador, na próxima verificação, o seu valor ainda é 1. A condição $1 \leq 5$ continua a ser verdadeira, e o ciclo repete-se para sempre.

Como Evitar e Corrigir

A prevenção é a melhor estratégia. Ao construir qualquer laço ENQUANTO ou FAÇA-ENQUANTO, o programador deve sempre seguir o checklist mental do "Ciclo de Vida do Laço":

1. **Inicialização:** "A minha variável de controle tem um valor inicial correto antes do laço começar?"
2. **Verificação:** "A minha condição lógica representa corretamente quando o laço deve parar?"
3. **Atualização:** "Dentro do laço, existe uma instrução que modifica a variável de controle e a aproxima da condição de parada?"

Se um programa parece estar "congelado" ou a repetir a mesma saída incessantemente, a causa mais provável é um loop infinito. Para depurar (corrigir) o problema, deve-se:

- Interromper a execução do programa.
- Inspeccionar o laço suspeito.
- Verificar se a variável da condição está a ser atualizada corretamente dentro do bloco do

laço.

Outros Erros Comuns

Além dos loops infinitos, outros erros podem ocorrer:

- **Erros de "Off-by-One":** Estes ocorrem quando o laço executa uma vez a mais ou uma vez a menos do que o esperado. Geralmente, são causados pelo uso incorreto de operadores de comparação. Por exemplo, usar < (menor que) quando se deveria usar <= (menor ou igual a) pode fazer com que a última iteração desejada não ocorra.
- **Condições Lógicas Complexas com Erros:** Em condições que envolvem múltiplos testes com operadores E e OU, um erro na lógica pode fazer com que o laço termine prematuramente ou continue por mais tempo do que o pretendido. É crucial testar estas condições cuidadosamente.

Adotar a prática de rever e testar mentalmente o fluxo de cada laço antes de executar o programa pode poupar muito tempo e frustração, construindo uma base sólida para a escrita de algoritmos robustos e corretos.

Capítulo 6:

Mãos à Obra: Lista de Exercícios Práticos

A teoria é fundamental, mas a verdadeira aprendizagem em programação acontece com a prática. Esta secção apresenta uma lista de exercícios com dificuldade progressiva para aplicar e solidificar os conceitos das estruturas ENQUANTO e FAÇA-ENQUANTO.

Nível 1: Fixação (Foco no ENQUANTO)

Estes exercícios visam reforçar a sintaxe básica e a lógica dos contadores.

1. **Contagem Crescente:** Escreva um algoritmo que utilize um laço ENQUANTO para imprimir na tela todos os números inteiros de 1 a 10.
2. **Contagem de Pares:** Escreva um algoritmo que imprima todos os números pares entre 1 e 20. (Dica: pode inicializar um contador em 2 e incrementá-lo de 2 em 2, ou verificar se o número é par dentro do laço).
3. **Tabuada:** Crie um algoritmo que peça ao utilizador para digitar um número inteiro. Em seguida, utilize um laço ENQUANTO para calcular e imprimir a tabuada desse número, do 1 ao 10. (Ex: Se o utilizador digitar 7, a saída deve ser "7 x 1 = 7", "7 x 2 = 14", ..., "7 x 10 = 70").

Nível 2: Aplicação (Contadores, Acumuladores e Condicionais)

Estes exercícios combinam laços com lógica condicional e os padrões de acumulador e contador.

4. **Soma de Valores:** Escreva um algoritmo que leia 10 números inteiros digitados pelo utilizador e, ao final, apresente a soma de todos os números lidos.
5. **Média de Notas:** Crie um algoritmo que leia 5 notas de um aluno, calcule e exiba a média final. Utilize um laço para ler as notas e um acumulador para somá-las.
6. **Contador com Sentinela:** Faça um algoritmo que leia números inteiros do teclado. O programa deve ler os números até que o utilizador digite um número negativo. No final, exiba quantos números positivos foram digitados.

Nível 3:

Desafio (Foco no FAÇA-ENQUANTO e Lógica Complexa)

Estes problemas são ideais para a estrutura FAÇA-ENQUANTO e exigem uma lógica mais elaborada.

7. **Validação de Dados:** Faça um programa que peça ao utilizador para digitar uma nota entre zero e dez. Se o utilizador digitar um valor inválido, o programa deve mostrar uma mensagem de erro e pedir a nota novamente. O programa só deve terminar quando uma nota válida for inserida. (Dica: Este é o cenário perfeito para um laço FAÇA-ENQUANTO).

8. **Menu de Calculadora:** Crie um programa que funcione como uma calculadora simples.

Ele deve apresentar o seguinte menu:

- 1 - Somar
- 2 - Subtrair
- 3 - Multiplicar
- 4 - Dividir
- 0 - Sair

O programa deve permitir que o utilizador escolha uma opção, insira dois números e veja o resultado. O menu deve ser exibido repetidamente até que o utilizador escolha a opção 0 para sair.

9. **Simulação de Crescimento:** Chico tem 1,50m de altura e cresce 2 centímetros por ano, enquanto Juca tem 1,10m de altura e cresce 3 centímetros por ano. Construa um algoritmo que calcule e imprima quantos anos serão necessários para que Juca seja mais alto que Chico. (Dica: Utilize um laço e variáveis para simular a passagem de cada ano).

Apêndice

A. Glossário de Termos Essenciais

- **Laço (Loop):** Uma estrutura de controle de programação que permite executar um bloco de código repetidamente.
- **Iteração:** Refere-se a uma única passagem ou execução do bloco de código dentro de um laço.
- **Condição:** Uma expressão lógica (que avalia para verdadeiro ou falso) que determina se um laço deve continuar a executar ou deve parar.
- **Contador:** Uma variável, tipicamente numérica, usada para contar o número de iterações de um laço ou a ocorrência de um evento.
- **Acumulador:** Uma variável usada para coletar ou somar valores ao longo das iterações de um laço, como calcular uma soma total ou um produto.
- **Sentinela:** Um valor específico, pré-definido, que sinaliza o fim de uma entrada de dados, atuando como uma condição de parada para um laço. Por exemplo, digitar 0 ou um número negativo para parar de somar valores.
- **Loop Infinito:** Um laço cuja condição de parada nunca é satisfeita, fazendo com que ele execute indefinidamente e "congele" o programa. Geralmente é causado por um erro de lógica.

B. Guia Rápido de Sintaxe e Fluxogramas

Estrutura	ENQUANTO (Pré-teste)	FAÇA-ENQUANTO (Pós-teste)
Sintaxe (Portugol)	ENQUANTO <condição> FACA // Bloco de comandos FIM_ENQUANTO	FAÇA // Bloco de comandos ENQUANTO <condição>
Fluxograma	O teste (losango) vem antes do bloco de ação (retângulo).	O bloco de ação (retângulo) vem antes do teste (losango).
Execução Mínima	0 vezes	1 vez

C. Dicas para Depuração ("Caça aos Erros")

Encontrar erros (bugs) no código é uma parte normal do processo de programação. Aqui estão algumas técnicas para encontrar e corrigir problemas nos seus laços:

- **Teste de Mesa (Execução Manual):** Esta é a técnica mais poderosa para um iniciante. Pegue um papel e uma caneta. Crie colunas para cada variável importante no seu algoritmo. Siga o código linha por linha, como se fosse o computador, e anote a mudança no valor de cada variável a cada passo (iteração). Isto ajuda a visualizar exatamente o que o programa está a fazer e a identificar onde a lógica falha.¹⁰
- **Mensagens de Depuração:** Adicione comandos escreva() ou escreval() temporários dentro do seu laço para imprimir o valor das suas variáveis de controle (contadores, acumuladores, etc.) a cada iteração. Ver os valores em tempo real pode revelar rapidamente por que um laço não está a comportar-se como esperado.
- **Verifique o "Ciclo de Vida":** Se o seu laço está em loop infinito ou não executa quando deveria, reveja sistematicamente as três etapas:

1. **Inicialização:** A variável de controle foi inicializada corretamente antes do laço?
2. **Verificação:** A condição está escrita corretamente? Usou < em vez de <= por engano?
3. **Atualização:** Existe uma linha *dentro* do laço que altera a variável de controle, aproximando-a da condição de parada?